29

基于锁的并发数据结构

在讨论锁之外的话题之前，我们先介绍如何在一些常见的数据结构中使用锁。给数据结构加上锁，让多个线程都能使用它，这个数据结构就成了**线程安全**（thread safe）的。当然，锁具体怎么加，既决定了数据结构是否正确，也决定了它的性能。于是，我们面临的挑战是：

> 关键问题：如何给数据结构加锁
> 面对一个具体的数据结构，应该怎样给它加锁，才能让它正确工作？进一步说，怎样加锁才能让这个数据结构有较高的性能，允许许多线程同时访问它，也就是**并发**（concurrently）访问？

当然，我们不可能涵盖所有数据结构，也不可能涵盖所有增加并发的方法，因为这个课题已经研究了很多年，发表的研究论文（毫不夸张地说）数以千计。所以，我们只打算把需要的思考方式讲到足以入门的程度，再给你推荐一些不错的资料，供你自己深入研究。我们发现，Moir 和 Shavit 的综述是很好的信息来源 [MS04]。

29.1 并发计数器

计数器是最简单的数据结构之一。它很常用，接口也很简单。图 29.1 定义了一个简单的非并发计数器。

简单但无法扩展

可以看到，这个没有同步的计数器是个再简单不过的数据结构，只要很少的代码就能实现。现在我们面临下一个挑战：怎样让这段代码变成**线程安全**的？图 29.2 展示了我们的做法。

这个并发计数器很简单，而且能正确工作。实际上，它沿用了最简单、最基本的并发数据结构（concurrent data structure）常用的一种设计模式：只加一把锁，调用操作这个数据结构的例程时获取锁，从调用返回时释放锁。从这一点看，它和用**管程**（monitor）构建的数据结构 [BH73] 很像：在管程中，调用对象方法和从方法返回时，锁会自动获取和释放。

```
1  typedef struct __counter_t {
2      int value;
3  } counter_t;
4
5  void init(counter_t *c) {
6      c->value = 0;
7  }
8
9  void increment(counter_t *c) {
10     c->value++;
11 }
12
13 void decrement(counter_t *c) {
14     c->value--;
15 }
16
17 int get(counter_t *c) {
18     return c->value;
19 }
```

图 29.1：**不带锁的计数器**

到这里，你已经有了一个能用的并发数据结构。你可能会遇到的问题是性能。如果数据结构太慢，光加一把锁就不够了；如果确实需要优化，本章剩下的部分讲的就是这些优化。注意，如果数据结构不算太慢，那就大功告成了！简单的办法能行，就没必要去搞花哨的东西。

为了弄清这种简单方法的性能代价，我们运行了一个基准测试：每个线程把同一个共享计数器更新固定的次数，然后改变线程的数量。图 29.5 给出了 1 到 4 个线程运行时花费的总时间，每个线程都把计数器更新一百万次。这个实验是在一台装有 4 个 2.7 GHz Intel i5 CPU 的 iMac 上运行的；参与运行的 CPU 越多，我们希望单位时间内完成的总工作量也越多。

从图中上面那条线（标为“精确”）可以看出，这个同步计数器的扩展性很差。单个线程只要很短的时间（大约 0.03 秒）就能完成一百万次更新，而让两个线程各自并发地把计数器更新一百万次，速度就会大幅下降（要花 5 秒多！）。线程越多，情况越糟。

理想情况下，你会希望多个线程在多个处理器上完成工作的速度，和单个线程在一个处理器上一样快。达到这个目标叫作**完美扩展**（perfect scaling）：虽然完成的工作更多了，但它们是并行完成的，所以完成任务所花的时间并不会增加。

可扩展的计数

令人惊讶的是，研究人员多年来一直在研究如何构建扩展性更好的计数器 [MS04]。更令人惊讶的是，可扩展的计数器确实很重要，最近关于操作系统性能分析的研究就表明了这一点 [B+10]：如果没有可扩展的计数，一些运行在 Linux 上的工作负载在多核机器上会遇到严重的扩展性问题。

```
1  typedef struct __counter_t {
2      int             value;
3      pthread_mutex_t lock;
4  } counter_t;
5
6  void init(counter_t *c) {
7      c->value = 0;
8      Pthread_mutex_init(&c->lock, NULL);
9  }
10
11 void increment(counter_t *c) {
12     Pthread_mutex_lock(&c->lock);
13     c->value++;
14     Pthread_mutex_unlock(&c->lock);
15 }
16
17 void decrement(counter_t *c) {
18     Pthread_mutex_lock(&c->lock);
19     c->value--;
20     Pthread_mutex_unlock(&c->lock);
21 }
22
23 int get(counter_t *c) {
24     Pthread_mutex_lock(&c->lock);
25     int rc = c->value;
26     Pthread_mutex_unlock(&c->lock);
27     return rc;
28 }
```

图 29.2：**带锁的计数器**

人们已经开发出许多技术来解决这个问题。我们介绍其中一种，叫作**近似计数器**（approximate counter）[C06]。

近似计数器用许多局部的物理计数器（每个 CPU 核一个）加上一个全局计数器，来表示一个逻辑计数器。具体来说，在一台有 4 个 CPU 的机器上，有 4 个局部计数器和 1 个全局计数器。除了这些计数器，还有锁：每个局部计数器各有一把锁[1]，全局计数器也有一把。

近似计数的基本思路如下。运行在某个核上的线程想增加计数器时，就增加它的局部计数器；对这个局部计数器的访问，通过对应的局部锁来同步。由于每个 CPU 都有自己的局部计数器，不同 CPU 上的线程可以互不竞争地更新各自的局部计数器，所以计数器的更新是可扩展的。

不过，为了让全局计数器保持最新（以防有线程要读它的值），局部计数器的值会定期转移到全局计数器上：先获取全局锁，把全局计数器加上局部计数器的值，然后把局部计数器清零。

[1]之所以需要局部锁，是因为我们假设每个核上可能不止一个线程。反过来，如果每个核上只运行一个线程，就不需要局部锁了。

时间	L_1	L_2	L_3	L_4	G
0	0	0	0	0	0
1	0	0	1	1	0
2	1	0	2	1	0
3	2	0	3	1	0
4	3	0	3	2	0
5	4	1	3	3	0
6	$5 \rightarrow 0$	1	3	4	5（来自 L_1）
7	0	2	4	$5 \rightarrow 0$	10（来自 L_4）

图 29.3：**跟踪近似计数器**

这种从局部到全局的转移多久发生一次，由阈值 S 决定。S 越小，计数器的表现就越像前面那个不可扩展的计数器；S 越大，计数器的扩展性越好，但全局计数器的值和实际计数的偏差也可能越大。当然也可以获取所有局部锁和全局锁（按规定的顺序获取，以免死锁），得到一个精确值，但这样做是不可扩展的。

为了说明清楚，我们来看一个例子（图 29.3）。在这个例子中，阈值 S 设为 5，4 个 CPU 上各有线程在更新自己的局部计数器 $L_1 \cdots L_4$。跟踪记录里也给出了全局计数器的值（G），时间从上往下递增。每个时间步，某个局部计数器可能会加一；如果局部计数器的值达到阈值 S，就把它转移到全局计数器，并把局部计数器清零。

图 29.5（第 6 页）中下面那条线（标为“近似”）是阈值 S 为 1024 时近似计数器的性能。性能非常好：在 4 个处理器上把计数器更新四百万次所花的时间，几乎不比在 1 个处理器上更新一百万次多。

图 29.6 说明了阈值 S 的重要性：4 个线程在 4 个 CPU 上各把计数器增加一百万次。如果 S 很小，性能就很差（但全局计数总是相当准确）；如果 S 很大，性能非常好，但全局计数会滞后（最多滞后 CPU 数乘以 S）。近似计数器提供的，正是这种准确性和性能之间的权衡。

图 29.4（第 5 页）给出了近似计数器的一个粗略实现。读一读它；更好的办法是自己动手，用它做些实验，以便更好地理解它是怎么工作的。

29.2 并发链表

接下来我们研究一种更复杂的结构：链表。我们还是先从一个基本的方法开始。为简单起见，我们省略这种链表显然该有的一些例程，只关注并发的插入和查找；删除等操作留给读者自己思考。图 29.7 是这个初级数据结构的代码。

从代码中可以看到，插入例程只是在进入时获取锁，退出时释放锁。如果 `malloc()` 碰巧失败（这种情况很少见），就会出现一个有点棘手的小问题：这时代码还必须在插入失败返回之前释放锁。

事实证明，这种异常的控制流很容易出错：最近一项针对 Linux 内核补丁的研究发现，有很大一部分 bug（将近 40%）出现在这类很少执行的代码路径上（实际上，这个发现促

```
typedef struct __counter_t {
    int             global;          // 全局计数
    pthread_mutex_t glock;           // 全局锁
    int             local[NUMCPUS];  // 每个 CPU 的计数
    pthread_mutex_t llock[NUMCPUS];  // ……以及对应的锁
    int             threshold;       // 更新频率
} counter_t;

// init：记录阈值，初始化各个锁，
//       以及所有局部计数和全局计数的值
void init(counter_t *c, int threshold) {
    c->threshold = threshold;
    c->global = 0;
    pthread_mutex_init(&c->glock, NULL);
    int i;
    for (i = 0; i < NUMCPUS; i++) {
        c->local[i] = 0;
        pthread_mutex_init(&c->llock[i], NULL);
    }
}

// update：通常只需获取局部锁，更新局部计数；
// 局部计数一旦涨到 threshold，
// 就获取全局锁，把局部计数的值转移过去
void update(counter_t *c, int threadID, int amt) {
    int cpu = threadID % NUMCPUS;
    pthread_mutex_lock(&c->llock[cpu]);
    c->local[cpu] += amt;
    if (c->local[cpu] >= c->threshold) {
        // 转移到全局计数（假定 amt>0）
        pthread_mutex_lock(&c->glock);
        c->global += c->local[cpu];
        pthread_mutex_unlock(&c->glock);
        c->local[cpu] = 0;
    }
    pthread_mutex_unlock(&c->llock[cpu]);
}

// get：直接返回全局计数（近似值）
int get(counter_t *c) {
    pthread_mutex_lock(&c->glock);
    int val = c->global;
    pthread_mutex_unlock(&c->glock);
    return val; // 只是近似值!
}
```

图 29.4：**近似计数器的实现**

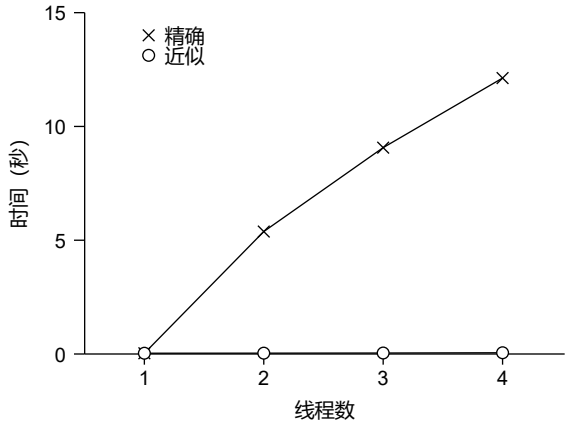


图 29.5：**传统计数器与近似计数器的性能对比**

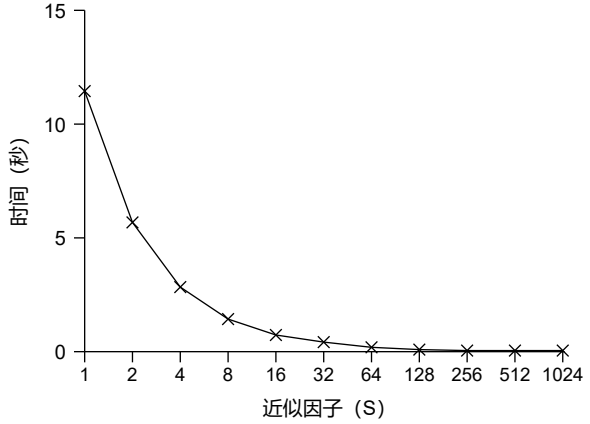


图 29.6：**近似计数器的扩展性**

提示：并发度更高不一定更快

如果你设计的方案增加了大量开销（比如频繁地获取和释放锁，而不是只做一次），那么它的并发度更高这一点可能就无关紧要了。简单的方案往往效果很好，尤其是很少调用代价高昂的例程时。加更多的锁、增加复杂度，反而可能让你栽跟头。话虽如此，要想真正知道答案，只有一个办法：把两种方案都实现出来（简单但并发度低的，和复杂但并发度高的），然后测一测它们的表现。说到底，性能是作不了弊的：你的想法要么更快，要么不快。

```
// 基本的节点结构
typedef struct __node_t {
    int              key;
    struct __node_t  *next;
} node_t;

// 基本的链表结构（每个链表用一个）
typedef struct __list_t {
    node_t           *head;
    pthread_mutex_t  lock;
} list_t;

void List_Init(list_t *L) {
    L->head = NULL;
    pthread_mutex_init(&L->lock, NULL);
}

int List_Insert(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *new = malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&L->lock);
        return -1; // 失败
    }
    new->key  = key;
    new->next = L->head;
    L->head   = new;
    pthread_mutex_unlock(&L->lock);
    return 0; // 成功
}

int List_Lookup(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *curr = L->head;
    while (curr) {
        if (curr->key == key) {
            pthread_mutex_unlock(&L->lock);
            return 0; // 成功
        }
        curr = curr->next;
    }
    pthread_mutex_unlock(&L->lock);
    return -1; // 失败
}
```

图 29.7：**并发链表**

成了我们自己的一些研究：我们从一个 Linux 文件系统中去掉了所有可能因内存分配失败而出错的路径，让系统更加健壮 [S+11]）。

于是挑战来了：能不能重写插入和查找例程，让它们在并发插入时依然正确，同时又不必在失败路径上也加一次解锁调用？

在这个例子里，答案是可以。具体来说，我们可以稍微调整一下代码：让插入代码中的加锁和解锁只包住真正的临界区（critical section），并让查找代码使用一条共同的退出路径。前一种改法之所以可行，是因为插入操作有一部分其实不需要加锁；假设 `malloc()` 本身是线程安全的，每个线程都可以放心地调用它，不必担心竞态条件（race condition，也叫竞争条件）或其他并发 bug。只有在更新共享的链表时才需要持有锁。这些修改的细节见图 29.8。

至于查找例程，只需做一个简单的代码变换：从主搜索循环跳出来，走到唯一的一条返回路径。这样做同样减少了代码中获取和释放锁的地方，从而降低了不小心引入 bug（比如返回前忘了解锁）的可能性。

提高链表的可扩展性

虽然我们又得到了一个基本的并发链表，但它的扩展性还是不太好。为了让链表内部支持更多的并发，研究人员探索过一种叫作**交替加锁**（hand-over-hand locking）的技术，也叫**锁耦合**（lock coupling）[MS04]。

思路很简单。不再给整个链表只配一把锁，而是给链表的每个节点各加一把锁。遍历链表时，代码先获取下一个节点的锁，再释放当前节点的锁（英文名 hand-over-hand 就是由此而来）。

从概念上讲，交替加锁的链表有一定道理：它让链表操作可以高度并发。但在实践中，很难让这种结构比简单的单锁方法更快，因为遍历链表时为每个节点获取和释放锁，开销大得让人难以接受。即使链表非常长、线程非常多，允许多个遍历同时进行所带来的并发，也不太可能比直接获取一把锁、执行操作、再释放它更快。也许某种混合方案（每隔若干个节点才获取一把新锁）值得研究一下。

29.3 并发队列

现在你已经知道，构造并发数据结构总有一种标准方法：加一把大锁。队列的这种做法我们就跳过了，相信你自己能想出来。

我们来看一个并发度稍高一些的队列，它是由 Michael 和 Scott 设计的 [MS98]。这个队列用到的数据结构和代码见图 29.9（第 10 页）。

仔细研究这段代码，你会发现它有两把锁：一把用于队列的头部，一把用于尾部。这两把锁的目的是让入队和出队操作能够并发进行。通常情况下，入队例程只访问尾锁，出队例程只访问头锁。

Michael 和 Scott 用到的一个技巧是加入一个哑节点（dummy node，在队列初始化代码中分配）；有了这个哑节点，头部操作和尾部操作就分开了。研究一下这段代码，最好是自己敲进去、运行并测量一下，深入理解它是怎么工作的。

```
1  void List_Init(list_t *L) {
2      L->head = NULL;
3      pthread_mutex_init(&L->lock, NULL);
4  }
5
6  int List_Insert(list_t *L, int key) {
7      // 这里不需要同步
8      node_t *new = malloc(sizeof(node_t));
9      if (new == NULL) {
10         perror("malloc");
11         return -1;
12     }
13     new->key = key;
14     // 只锁住临界区
15     pthread_mutex_lock(&L->lock);
16     new->next = L->head;
17     L->head   = new;
18     pthread_mutex_unlock(&L->lock);
19     return 0; // 成功
20 }
21
22 int List_Lookup(list_t *L, int key) {
23     int rv = -1;
24     pthread_mutex_lock(&L->lock);
25     node_t *curr = L->head;
26     while (curr) {
27         if (curr->key == key) {
28             rv = 0;
29             break;
30         }
31         curr = curr->next;
32     }
33     pthread_mutex_unlock(&L->lock);
34     return rv; // 现在成功和失败都走这里
35 }
```

图 29.8: **重写后的并发链表**

提示：小心锁和控制流

有一条通用的设计建议，在并发代码和其他地方都适用：要小心那些会导致函数返回、退出，或者因其他类似错误而中止函数执行的控制流变化。很多函数一开始会获取锁、分配内存，或者做其他类似的有状态的操作，一旦出错，代码就得在返回前把这些状态全部撤销，而这很容易出错。因此，组织代码时最好尽量避免这种模式。

```
1  typedef struct __node_t {
2      int              value;
3      struct __node_t  *next;
4  } node_t;
5
6  typedef struct __queue_t {
7      node_t           *head;
8      node_t           *tail;
9      pthread_mutex_t  head_lock, tail_lock;
10 } queue_t;
11
12 void Queue_Init(queue_t *q) {
13     node_t *tmp = malloc(sizeof(node_t));
14     tmp->next = NULL;
15     q->head = q->tail = tmp;
16     pthread_mutex_init(&q->head_lock, NULL);
17     pthread_mutex_init(&q->tail_lock, NULL);
18 }
19
20 void Queue_Enqueue(queue_t *q, int value) {
21     node_t *tmp = malloc(sizeof(node_t));
22     assert(tmp != NULL);
23     tmp->value = value;
24     tmp->next  = NULL;
25
26     pthread_mutex_lock(&q->tail_lock);
27     q->tail->next = tmp;
28     q->tail = tmp;
29     pthread_mutex_unlock(&q->tail_lock);
30 }
31
32 int Queue_Dequeue(queue_t *q, int *value) {
33     pthread_mutex_lock(&q->head_lock);
34     node_t *tmp = q->head;
35     node_t *new_head = tmp->next;
36     if (new_head == NULL) {
37         pthread_mutex_unlock(&q->head_lock);
38         return -1; // 队列为空
39     }
40     *value = new_head->value;
41     q->head = new_head;
42     pthread_mutex_unlock(&q->head_lock);
43     free(tmp);
44     return 0;
45 }
```

图 29.9：**Michael 和 Scott 的并发队列**

```
1  #define BUCKETS (101)
2
3  typedef struct __hash_t {
4      list_t lists[BUCKETS];
5  } hash_t;
6
7  void Hash_Init(hash_t *H) {
8      int i;
9      for (i = 0; i < BUCKETS; i++)
10         List_Init(&H->lists[i]);
11 }
12
13 int Hash_Insert(hash_t *H, int key) {
14     return List_Insert(&H->lists[key % BUCKETS], key);
15 }
16
17 int Hash_Lookup(hash_t *H, int key) {
18     return List_Lookup(&H->lists[key % BUCKETS], key);
19 }
```

图 29.10：**并发散列表**

队列在多线程应用程序中很常用。然而，这里用的这种队列（只用了锁）往往不能完全满足这类程序的需要。还有一种更完善的有界队列，能让线程在队列为空或过满时等待，它是下一章（条件变量）要深入研究的主题。敬请期待！

29.4 并发散列表

最后，我们来讨论一种简单又应用广泛的并发数据结构：散列表。我们关注的是一个不会调整大小的简单散列表；处理调整大小还要多做一些工作，这就留给读者作为练习了（抱歉！）。

这个并发散列表（图 29.10）很直观，它是用前面开发的并发链表构建的，效果出奇地好。它性能好的原因在于，它没有给整个结构只配一把锁，而是给每个散列桶（hash bucket，每个桶用一个链表表示）各配一把锁。这样就能让许多并发操作同时进行。

图 29.11（第 12 页）展示了散列表在并发更新下的性能（4 个线程各自并发更新 10,000 到 50,000 次，还是在那台有 4 个 CPU 的 iMac 上）。为了对比，图中也给出了链表（只有一把锁）的性能。从图中可以看出，这个简单的并发散列表扩展得非常好；相比之下，链表则不然。

29.5 小结

我们介绍了一些并发数据结构的例子，从计数器，到链表和队列，最后是无处不在、用得非常多的散列表。一路上我们学到了几条重要的经验：在控制流发生变化的地方，获取和释放锁要格外小心；提高并发度不一定能提高性能；性能问题只应在真正出现之后再去解决。

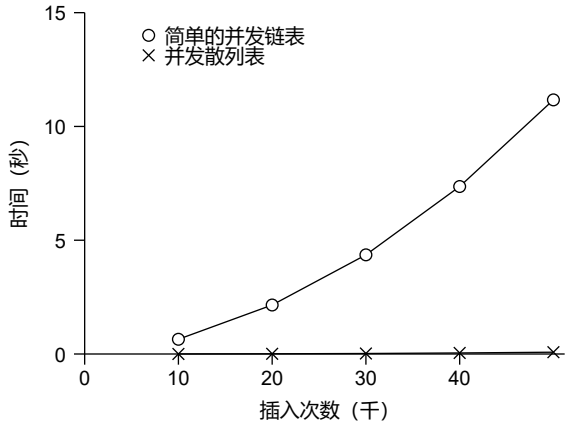


图 29.11：**散列表的扩展性**

最后这一点，也就是避免**过早优化**（premature optimization），对任何重视性能的开发者都至关重要：如果把某个部分做快了并不能提高应用程序的整体性能，那么这样做毫无价值。

当然，我们只是触及了高性能数据结构的皮毛。更多内容请参阅 Moir 和 Shavit 出色的综述，里面还有指向其他资料的链接 [MS04]。特别是，你可能会对其他结构（比如 B 树）感兴趣；想学这方面的知识，最好去上一门数据库课。你可能还会好奇那些完全不用传统锁的技术；这类**非阻塞数据结构**（non-blocking data structures），我们会在“常见并发问题”一章中稍作介绍，但坦白说，这个主题本身就是一整个知识领域，要学的东西远远超出了这本小书所能容纳的范围。如果愿意，你可以自己去深入了解（一如既往！）。

提示：避免过早优化（Knuth 定律）

构建并发数据结构时，先从最基本的方法开始，也就是加一把大锁来提供同步访问。这样，你很可能会构建出一把*正确*的锁；之后如果发现它有性能问题，再去改进它，也就是只在有必要时才让它变快。正如 **Knuth** 的名言所说：“过早优化是万恶之源。”

许多操作系统在最初过渡到多处理器时都只用了一把锁，Sun OS 和 Linux 都是如此。在 Linux 中，这把锁甚至有个名字，叫**大内核锁**（big kernel lock，**BKL**）。多年来，这种简单的方法一直很好用，但当多 CPU 系统成为常态后，同一时刻只允许内核中有一个活动线程，就成了性能瓶颈。于是，终于到了给这些系统加上“提高并发度”这项优化的时候。Linux 采用了比较直接的方法：把一把锁换成许多把锁。Sun 则做出了一个更激进的决定：构建一个全新的操作系统 Solaris，从第一天起就在更根本的层面上融入并发。想进一步了解这些引人入胜的系统，请阅读 Linux 和 Solaris 的内核书籍 [BC05, MM00]。

参考文献

[B+10] "An Analysis of Linux Scalability to Many Cores" by Silas Boyd-Wickizer, Austin T. Clements, Yandong Mao, Aleksey Pesterev, M. Frans Kaashoek, Robert Morris, Nickolai Zeldovich. OSDI '10, Vancouver, Canada, October 2010. *A great study of how Linux performs on multicore machines, as well as some simple solutions. Includes a neat* **sloppy counter** *to solve one form of the scalable counting problem.*

[BH73] "Operating System Principles" by Per Brinch Hansen. Prentice-Hall, 1973. Available: http://portal.acm.org/citation.cfm?id=540365. *One of the first books on operating systems; certainly ahead of its time. Introduced monitors as a concurrency primitive.*

[BC05] "Understanding the Linux Kernel (Third Edition)" by Daniel P. Bovet and Marco Cesati. O'Reilly Media, November 2005. *The classic book on the Linux kernel. You should read it.*

[C06] "The Search For Fast, Scalable Counters" by Jonathan Corbet. February 1, 2006. Available: https://lwn.net/Articles/170003. *LWN has many wonderful articles about the latest in Linux. This article is a short description of scalable approximate counting; read it, and others, to learn more about the latest in Linux.*

[L+13] "A Study of Linux File System Evolution" by Lanyue Lu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Shan Lu. FAST '13, San Jose, CA, February 2013. *Our paper that studies every patch to Linux file systems over nearly a decade. Lots of fun findings in there; read it to see! The work was painful to do though; the poor graduate student, Lanyue Lu,* had *to look through every single patch by hand in order to understand what they did.*

[MS98] "Nonblocking Algorithms and Preemption-safe Locking on Multiprogrammed Shared-memory Multiprocessors" by M. Michael, M. Scott. Journal of Parallel and Distributed Computing, Vol. 51, No. 1, 1998. *Professor Scott and his students have been at the forefront of concurrent algorithms and data structures for many years; check out his web page, numerous papers, or books to find out more.*

[MS04] "Concurrent Data Structures" by Mark Moir and Nir Shavit. In Handbook of Data Structures and Applications (Editors D. Metha and S.Sahni). Chapman and Hall/CRC Press, 2004. Available: www.ostep.org/Citations/concurrent.pdf. *A short but relatively comprehensive reference on concurrent data structures. Though it is missing some of the latest works in the area (due to its age), it remains an incredibly useful reference.*

[MM00] "Solaris Internals: Core Kernel Architecture" by Jim Mauro and Richard McDougall. Prentice Hall, October 2000. *The Solaris book. You should also read this, if you want to learn about something other than Linux.*

[S+11] "Making the Common Case the Only Case with Anticipatory Memory Allocation" by Swaminathan Sundararaman, Yupu Zhang, Sriram Subramanian, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '11, San Jose, CA, February 2011. *Our work on removing possibly-failing allocation calls from kernel code paths. By allocating all potentially needed memory before doing any work, we avoid failure deep down in the storage stack.*

作业（编程）

在这个作业中，你将积累一些编写并发代码、测量其性能的经验。学会写出性能良好的代码是一项关键技能，所以在这里积累一点经验很值得。

问题

1. 首先，我们来重做本章中的测量。在程序里用 `gettimeofday()` 调用来测量时间。这个计时器有多精确？它能测量的最小间隔是多少？要对它的工作方式心中有数，因为后面所有问题都要用到它。你也可以研究一下其他计时器，比如 x86 上通过 `rdtsc` 指令提供的周期计数器。
2. 现在，构建一个简单的并发计数器，测量随着线程数增加，把计数器增加很多次要花多长时间。你使用的系统上有几个 CPU？这个数目对你的测量结果有影响吗？
3. 接下来，实现一个近似计数器。同样，测量它的性能随线程数和阈值如何变化。这些数字和你在本章中看到的一致吗？
4. 按本章引用的文献 [MS04]，实现一个使用交替加锁的链表。你应该先读这篇论文，弄懂它的原理，然后再实现。测量它的性能。交替加锁的链表在什么情况下比本章中的标准链表表现更好？
5. 挑一个你喜欢的数据结构，比如 B 树，或者其他稍微有趣一点的结构。实现它，先采用简单的加锁策略，比如只用一把锁。测量随着并发线程数增加，它的性能如何变化。
6. 最后，为你喜欢的这个数据结构想一个更有意思的加锁策略。实现它，并测量它的性能。它和直截了当的加锁方法相比如何？